Week 14 Homework: From Tested Messages to a Classifier

Goal: understand logistic regression as one sigmoid neuron, extend the idea to an MLP, and evaluate both without implementing the training algorithm.

Files provided. Use `week14_messages.csv`, a synthetic teaching dataset of 100 demand-response messages. Each row contains message text, six input features, and a binary label called `useful`. The file is intentionally simplified and is not evidence about real household behavior.

AI policy. You may use AI or web search to understand official Python or `sklearn` documentation, ask conceptual questions, or interpret an error message. You may **not** ask AI to write the required logistic score, sigmoid, threshold rule, confusion-matrix counts, classification metrics, MLP forward pass, model-comparison analysis, or causal explanation. Complete all `TODO` sections manually. If you use AI, add one sentence stating exactly what it helped you understand.

Key reminders. For binary logistic regression,

$$z = b + w^T x, \qquad \hat{p} = \sigma(z) = \frac{1}{1 + e^{-z}}, \qquad \hat{y} = \mathbb{1}(\hat{p} \geq t).$$

A one-hidden-layer MLP makes a forward prediction using

$$h = \mathrm{ReLU}(W_1 x + b_1), \qquad \hat{p} = \sigma(W_2 h + b_2).$$

The library may train the weights with gradient-based optimization, but this homework does not ask you to implement or derive that training method.

Monday: From the Week 13 Experiment to Labels and Features

Week 13 asked whether a text message *causes* a reduction in household electricity use from 5–8 p.m. Week 14 assumes that 100 message trials—including some repeated templates under different conditions—have already been assigned labels.

1. In two or three sentences, explain the difference between the causal question "Did this message reduce use?" and the prediction question "Will this new message receive the useful label?"
2. Propose a precise experimental rule for assigning `useful = 1`. Include a practically meaningful reduction, a comparison group, and an uncertainty requirement.
3. Explain why testing one message on one household would not create a reliable label.
4. Name two reasons that experimental labels could be noisy or biased.
5. For each feature below, state whether it is binary categorical or numeric:
   - `has_savings`, `has_action`, `has_deadline`, `has_social_norm`;
   - `message_length`, `send_hour`.
6. Explain one advantage of human-designed keyword features and one advantage of a text embedding.
7. Explain why measured electricity reduction and the experiment's p-value must not be included as inputs when the target label was created from those same quantities.

Create a Python file named `week14_message_classifier.py`. Begin with the following data inspection.

```
import numpy as np
import pandas as pd

df = pd.read_csv("week14_messages.csv")

print(df.head())
print(df.shape)
print(df["useful"].value_counts())
print(df.groupby("useful")[[
    "has_savings", "has_action", "has_deadline",
    "has_social_norm", "message_length", "send_hour"
]].mean())

# TODO: Print the five shortest messages and their labels.
# TODO: Print every row with has_action == 1 and has_savings == 0.
# TODO: Count duplicated message texts with df["text"].duplicated().sum().
```

In one paragraph, explain what the group means suggest. Do not claim that any feature *causes* usefulness; the table shows association in synthetic data.

Tuesday: Manually Implement One Logistic Neuron

Use four binary inputs in this order:

$$x = [\text{savings}, \text{action}, \text{deadline}, \text{social norm}]^T.$$

Suppose a trained logistic model has

$$w = [1.2, 1.5, 0.8, 0.4]^T, \qquad b = -1.6.$$

By hand, calculate z, $\hat{p}$, and the predicted class at threshold $t = 0.5$ for:

$$x_A = [1, 1, 0, 0]^T, \quad x_B = [0, 1, 1, 1]^T, \quad x_C = [1, 0, 0, 0]^T.$$

Show every weighted-sum calculation and round probabilities to three decimal places.

Add the following code and complete every `TODO` manually.

```
def sigmoid(z):
    # TODO: return 1 / (1 + exp(-z)) using np.exp
    pass

def logistic_probability(x, w, b):
    # TODO: start the score at b
    # TODO: use a loop to add w[j] * x[j] for every feature
    # TODO: return sigmoid(score)
    pass

def classify_probability(probability, threshold=0.5):
    # TODO: return 1 when probability >= threshold; otherwise return 0
    pass

w = np.array([1.2, 1.5, 0.8, 0.4])
b = -1.6

examples = {
    "A": np.array([1.0, 1.0, 0.0, 0.0]),
    "B": np.array([0.0, 1.0, 1.0, 1.0]),
    "C": np.array([1.0, 0.0, 0.0, 0.0]),
}

for name, x in examples.items():
    # TODO: calculate probability and predicted class
    # TODO: print the name, probability, and class
    pass
```

Manual coding rule. Do not use `sklearn`, `np.dot`, the `@` operator, or an AI-written solution for these three functions. The purpose is to see the single neuron's weighted sum, sigmoid, and threshold explicitly.

1. Confirm that the code matches all three hand calculations.
2. Explain why the raw score z cannot directly be interpreted as a probability.
3. Explain, using the code, why logistic regression can be regarded as one sigmoid neuron with no hidden layer.
4. Repeat the classifications using thresholds 0.3, 0.5, and 0.8. State which predictions change.

Wednesday: Confusion Matrix and Threshold Tradeoffs

Suppose a test set has the following true labels and predicted probabilities.

```
y_true = np.array([1, 0, 1, 1, 0, 0, 1, 0, 1, 0])
y_prob = np.array([.91, .72, .64, .42, .38, .81, .55, .20, .49, .10])
```

1. At threshold 0.5, write the predicted-label vector by hand.
2. Count TP, TN, FP, and FN by hand.
3. Calculate accuracy, precision, recall, and F1-score by hand.

4. Repeat all calculations at threshold 0.7.
5. Which threshold has higher precision? Which has higher recall? Explain why.
6. If every false positive requires an expensive field experiment, which threshold would you prefer? State one reason the other threshold could still be defensible.

Complete these functions without calling `sklearn.metrics`.

```
def confusion_counts(y_true, y_pred):
    tp = 0
    tn = 0
    fp = 0
    fn = 0
    for truth, prediction in zip(y_true, y_pred):
        # TODO: update exactly one count for this example
        pass
    return tp, tn, fp, fn

def classification_metrics(tp, tn, fp, fn):
    # TODO: calculate accuracy
    # TODO: calculate precision
    # TODO: calculate recall
    # TODO: calculate F1 from precision and recall
    # You may assume the denominators are nonzero for this dataset.
    pass

for threshold in [0.3, 0.5, 0.7, 0.8]:
    # TODO: turn every probability into a class using your threshold function
    # TODO: calculate the four counts and four metrics
    # TODO: print one clearly labeled result line
    pass
```

Add a six-row table to your submission with one row per threshold from 0.30 through 0.80 in increments of 0.10. Include TP, FP, FN, precision, recall, and F1. Briefly describe the tradeoff you observe.

Thursday: Manually Run a Small MLP Forward Pass

The MLP below has four inputs, three ReLU hidden neurons, and one sigmoid output neuron. The weights have already been trained; your task is only to make a forward prediction.

```
W1 = np.array([
    [ 0.8,  0.7, -0.2,  0.1],
    [-0.4,  0.9,  0.8,  0.6],
    [ 0.5, -0.3,  0.4,  0.9],
])
b1 = np.array([-0.6, -0.7, -0.5])
W2 = np.array([1.1, 0.9, 0.7])
b2 = -0.8

def relu(z):
    # TODO: return an array whose negative entries become 0
    pass

def mlp_probability(x, W1, b1, W2, b2):
    # TODO: calculate hidden_score = W1 @ x + b1
    # TODO: calculate hidden = relu(hidden_score)
    # TODO: calculate output_score = W2 @ hidden + b2
    # TODO: return sigmoid(output_score), hidden_score, and hidden
    pass

for name, x in examples.items():
    # TODO: call mlp_probability
    # TODO: print the hidden scores, hidden activations, and final probability
    pass
```

For example A, calculate all three hidden scores, all three ReLU activations, the output score, and the sigmoid probability by hand. Then answer:

1. What did the hidden layer add between the original inputs and the final probability?
2. Why can different hidden neurons detect different feature combinations?
3. Why is this forward pass not the same as training?
4. Why might this MLP overfit a dataset containing only 100 messages?

Friday: Train with a Library, Compare, and Make a Research Decision

In this section, you may use `sklearn` to train the models. You are using an existing training implementation; you are not implementing gradient descent or backpropagation.

Add the following code and fill the `TODO` sections.

```
from sklearn.model_selection import train_test_split
from sklearn.pipeline import make_pipeline
from sklearn.preprocessing import StandardScaler
from sklearn.linear_model import LogisticRegression
from sklearn.neural_network import MLPClassifier

feature_cols = [
    "has_savings", "has_action", "has_deadline",
    "has_social_norm", "message_length", "send_hour"
]

# TODO: create X from feature_cols and y from the useful column

X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.20, random_state=14, stratify=y
)

logistic = make_pipeline(
    StandardScaler(),
    LogisticRegression(max_iter=2000)
)

mlp = make_pipeline(
    StandardScaler(),
    MLPClassifier(
        hidden_layer_sizes=(6,),
        max_iter=3000,
        random_state=14
    )
)

models = {"logistic": logistic, "MLP": mlp}

for name, model in models.items():
    # TODO: fit the model on X_train and y_train
    # TODO: obtain useful-class probabilities for X_test
    # TODO: classify at threshold 0.5
    # TODO: use your own functions to calculate counts and metrics
    # TODO: print every result clearly
    pass
```

1. Report the exact test-set size and class counts.
2. Compare logistic regression and the MLP using accuracy, precision, recall, and F1.
3. Change the MLP hidden layer from 6 neurons to 2 and then 20 neurons. Record the test metrics for all three versions.
4. Explain why one 20-message test split is not enough to declare a universal winner.

5. Choose the model you would use to prioritize the next five messages for field testing. Defend the choice using performance, interpretability, data size, and overfitting risk.
6. Explain why even the chosen classifier cannot establish that a newly flagged message will *cause* lower electricity use.
7. Propose the next controlled experiment after the model nominates five candidate messages.

Optional text-representation extension. Describe, without calling a paid API, how you would replace or supplement the six supplied features with (a) TF–IDF vectors or (b) precomputed text embeddings. State one advantage, one risk, and the expected shape of the resulting input matrix.

Feynman Technique Post

Write a beginner-friendly learning note titled "A Logistic Classifier Is One Neuron." It must:

1. begin with the Week 13 text-message experiment;
2. explain how 100 tested messages become labeled examples;
3. show one weighted-score and sigmoid calculation;
4. explain logistic regression as one neuron with no hidden layer;
5. explain what hidden neurons add in an MLP;
6. define precision and recall using the message-screening example;
7. include one hand calculation, diagram, table, or code-output screenshot; and
8. end by explaining why prediction does not replace a randomized experiment.

Submission checklist. Monday causal-versus-prediction explanation and data audit; Tuesday hand calculations and manual logistic functions; Wednesday confusion matrix, metrics, and threshold table; Thursday hand calculation and MLP forward-pass code; Friday library comparison and research decision; Feynman Technique post; one sentence disclosing any permitted AI help.